

QUIDDITY INNOVATIONS

MapSnap

Understand any enterprise database, in plain English

Components, Architecture & Open-Source Licensing

Technical reference for IT staff

A self-contained, local-first schema-intelligence tool: it maps any SQL Server / PostgreSQL / MySQL / Oracle / SQLite database into a navigable HTML browser and layers a local-AI assistant on top that explains the schema and turns plain-English questions into read-only SQL.

Python · localhost:9876 · local Ollama AI · compliance-by-construction

Generated 2026-06-27 · Source: QI Hive Project-Status library

1. What MapSnap is, for IT

In one line. MapSnap points at a large, often-undocumented enterprise database and produces a fully navigable map of every table, column, key and relationship, then adds an AI assistant that explains the schema and writes read-only SQL — with regulated data never leaving the machine by default.

Runtime shape. A single Python process runs a standard-library HTTP server on 127.0.0.1:9876 and serves one rich HTML application. There is no web framework (no Flask/FastAPI), no database server to install, and no mandatory cloud dependency. This keeps the install dependency-light and trivially portable to a customer or air-gapped box.

Why IT should care. MapSnap separates reasoning from data: a model reasons over schema structure and drafts SQL; the SQL executes locally; result rows render locally and never leave the box. A fail-safe egress guardrail (default policy `local_only`) is the single chokepoint for any cloud call, so FERPA/GLBA/HIPAA data cannot be casually shipped to a cloud AI.

Lineage. MapSnap is the design ancestor of CogniBase; the same engine-agnostic pipeline is intended to carry, unchanged, to a Boston University OnBase deployment.

2. Runtime & deployment architecture

Concern	Detail
Language / runtime	Python (CPython) 3.11+ — sole implementation language for server, extractors, enrichment and doc generators
Web server	Python stdlib <code>http.server</code> (<code>ThreadingHTTPServer</code>), threaded for SSE streaming; <code>~50 /api/*</code> routes
Bind address / port	127.0.0.1:9876 (<code>--port</code> override)
Front-end	One generated single-page HTML/CSS/vanilla-JS file (<code>schema_browser.html</code> via <code>build_browser.py</code>)
App storage	SQLite (<code>mapsnap_users.db</code>) for users + sessions; per-profile state as JSON sidecars
AI (default)	Local Ollama model at 127.0.0.1:11434 — 100% local; <code>granite</code> , <code>qwen2.5-coder</code> , <code>sqlcoder</code> , <code>gemma</code>
AI (optional)	OpenRouter / direct provider APIs — only reachable when a profile's <code>data_policy</code> permits
Windows service	QI_MapSnap (server) + QI_MapSnapTunnel (tunnel) via NSSM, AppDirectory <code>C:\MapSnap</code>
Public URL	<code>https://mapsnap.quiddityinnovations.com</code> via named Cloudflare tunnel (<code>qi-mapsnap</code>)
Logs	Rotated logs in <code>C:\MapSnap\LOGS</code>
Customer/offline mode	<code>start.bat</code> auto-detects service vs manual; falls back to local Python + Cloudflare quick tunnel

3. Component inventory

Every component in the MapSnap stack, grouped by layer. The open-source third-party components and the obligations they carry are detailed in section 5.

Layer	Component	What it does / how MapSnap uses it	License
Runtime	Python (CPython) 3.11+	Single implementation language for the server, extractors, enrichment pipeline and	PSF

		doc generators.	
Web server	stdlib http.server (ThreadingHTTPServer)	Serves the HTML browser and ~50 /api/* routes; threaded for SSE streaming. No web framework.	PSF
Frontend	Single-page HTML / CSS / vanilla JS	schema_browser.html (built by build_browser.py): table/column/FK/diagram/data/chat/compare tabs; fetch + SSE.	Proprietary (QI)
App storage	SQLite (sqlite3)	User accounts + sessions (mapsnap_users.db). Per-profile state stored as JSON sidecars, not a DB.	Public domain
Schema store	JSON sidecar files	schema.json + annotations.json + BU_Catalog.json + profile.json + mapsnap_meta.json + enrich_status.json per profile.	N/A (data)
DB driver	pyodbc (SQL Server)	Live extract + read-only query execution against SQL Server (OnBase hsi schema, Jenzabar).	MIT-0
DB driver	psycopg2 (PostgreSQL)	PostgresExtractor live extract + query execution.	LGPL
DB driver	PyMySQL (MySQL)	MySQLExtractor live extract + query execution.	MIT
DB driver	oracledb (Oracle)	OracleExtractor live extract + query execution (thin mode, no Oracle client install).	Apache-2.0 / UPL
Vector store	ChromaDB	Local per-profile .index/ for Layer 5 semantic retrieval; also a ChromaExtractor source.	Apache-2.0
Vector store	Qdrant client	QdrantExtractor — treat a Qdrant collection as a browsable 'schema' source.	Apache-2.0
Local LLM	Ollama	Default reasoning + NL->SQL engine (granite3.1-dense, qwen2.5-coder, sqlcoder, gemma). 100% local.	MIT
Local embeddings	nomic-embed-text (via Ollama)	768-dim embeddings for Layer 5; runs locally so vectors never leave the box.	Apache-2.0
Cloud LLM gateway	OpenRouter (OpenAI-compatible)	Optional frontier-model access, gated by the per-profile data_policy guardrail.	Commercial API
Cloud LLM (direct)	Direct provider APIs (Anthropic / OpenAI-class)	Optional direct frontier models with per-provider SSE delta parsing; same guardrail.	Commercial API
Document parsing	python-docx	Reads .docx reference docs for RAG; generates schema confirmation report, admin guide, feature list.	MIT
Document parsing	PyPDF2 / pdfminer.six	Optional PDF text extraction for the document library (graceful fallback if absent).	BSD / MIT
Spreadsheet parsing	openpyxl	Reads .xlsx schema/relation exports (e.g. OnBase IST schema workbooks).	MIT
Config import	zipfile + xml (stdlib)	Unpacks OnBase .spk/.expk Studio packages (ZIP+XML) to extract DocType / keyword definitions.	PSF
Auth crypto	hmac / hashlib / secrets (stdlib)	Password hashing and session-token generation for login mode.	PSF
Process / service	NSSM	Runs QI_MapSnap + QI_MapSnapTunnel as auto-start Windows services with log rotation.	Public domain
Remote access	Cloudflare Tunnel (cloudflared)	Named tunnel exposing localhost:9876 as the public HTTPS URL; quick-tunnel fallback on customer boxes.	Apache-2.0 (client)

4. The enrichment layers (how components combine)

MapSnap's intelligence comes from a per-profile 'knowledge bundle' — a stack of enrichment layers, each a plain file stored beside the schema. This is where the components above earn their place.

Layer	Code / artifact	What it does	Built on
Layer 0 — Egress guardrail	guardrail.py	Framework-free single chokepoint for cloud egress. Resolves the active policy (fail-safe local_only), blocks calls under local_only, scrubs regulated values and strips LOCAL_ONLY-sentinel spans under cloud_metadata_only, and logs every decision. Wraps /api/chat and /api/nl2sql only; local execution and local models are never touched.	Python stdlib
Layer 1 — Schema extraction	db_extractors.py, extract_schema.py	Interchangeable Sqlite/Postgres/MySQL/Oracle extractors (+ Chroma/Qdrant) and a pyodbc SQL Server path, all emitting one canonical schema.json. Soft-FK inference adds relationships for schemas (like OnBase) that declare none.	pyodbc / psycopg2 / PyMySQL / oracledb
Layer 2 — Semantic annotations	annotations.json	Per-table purpose/role text wired into the AI prompt so the model understands meaning, not just shape.	JSON sidecar
Layer 3 — Business-value catalog	BU_Catalog.json	Business-name values (e.g. OnBase DocType names) from live OnBase or curated .expk config import.	JSON sidecar
Layer 4 — Data profiling	profile.json	Real, PII-masked data samples and semantic types so the AI knows the actual data in place.	JSON sidecar
Layer 5 — Semantic retrieval	semantic_index.py + ChromaDB	One nomic-embed-text embedding per table stored in a local ChromaDB index; query time interleaves keyword scoring with semantic matches, with a pure-keyword fallback if no index exists or Ollama is down.	ChromaDB + nomic-embed-text

Each enrichment artifact is a plain JSON file beside schema.json; an enrich.py orchestrator runs the chain idempotently (source-hash gated) and best-effort, triggered by extract / document import / config import.

5. Open-source components pulled from GitHub / public repositories

MapSnap ships dependency-light and installs database drivers on demand through a strict pip allow-list, so the app only pulls the driver a given connection needs. The third-party components below are free-to-use open-source projects. None are modified; each is used as an unmodified library or external service.

Component	Upstream	License	What it does	Obligation / note
Ollama	github.com/ollama/ollama	MIT	Local LLM runtime/server (localhost:11434) for reasoning and NL→SQL.	Permissive — attribution only. Runs as a separate local service.
ChromaDB	github.com/chroma-core/chroma	Apache-2.0	On-disk vector store for Layer 5 semantic retrieval.	Permissive — keep NOTICE/attribution. Used as an unmodified library.
nomic-embed-text	huggingface.co/nomic-ai	Apache-2.0	Local text-embedding model served via Ollama.	Permissive — model weights, attribution only.
pyodbc	github.com/mkleehammer/pyodbc	MIT-0	SQL Server connectivity for live extract + queries.	MIT-0 — no obligations (no attribution required).
psycopg2	github.com/psycopg/psycopg2	LGPL-3.0	PostgreSQL driver.	LGPL — fine as an unmodified, dynamically-used library; do not modify the library itself without sharing changes.
PyMySQL	github.com/PyMySQL/PyMySQL	MIT	MySQL/MariaDB driver.	Permissive — attribution only.
oracledb	github.com/oracle/python-oracledb	Apache-2.0 / UPL	Oracle driver in thin mode (no client install).	Permissive — keep NOTICE.
python-docx	github.com/python-openxml/python-docx	MIT	Read .docx for RAG; generate reports/guides.	Permissive — attribution only.
openpyxl	foss.heptapod.net/openpyxl	MIT	Read .xlsx schema/relation exports.	Permissive — attribution only.
PyPDF2 / pdfminer.six	github.com/py-pdf/pypdf · github.com/pdfminer/pdfminer.six	BSD / MIT	Optional PDF text extraction for the document library.	Permissive — attribution only; optional (graceful fallback).
Qdrant client	github.com/qdrant/qdrant-client	Apache-2.0	Optional Qdrant-collection-as-schema source.	Permissive — keep NOTICE.
cloudflared	github.com/cloudflare/cloudflared	Apache-2.0	Cloudflare Tunnel client for public HTTPS exposure.	Permissive (client). Cloudflare service governed by Cloudflare ToS.
NSSM	nssm.cc / git.nssm.cc	Public domain	Runs the server + tunnel as Windows services.	Public domain — no obligations.

6. Feature → component mapping

How the technologies tie to what a user actually sees and does in MapSnap.

User-facing feature	Components that power it	How
Map any database into a browser	stdlib http.server, build_browser.py, db_extractors.py, pyodbc/psycogp2/PyMySQL/oracledb	Connect → extract → one canonical schema.json → generated single-page HTML browser.
See joins that aren't declared	infer_soft_fks.py	Infers relationships from column-name conventions so an OnBase dump stops looking like disconnected islands.
Ask 'where are the checks?'	Ollama + nomic-embed-text + ChromaDB (Layer 5)	Local model + hybrid keyword/semantic retrieval map cryptic names (hsi.itemdata) to meaning.
Plain-English → SQL	Ollama (NL→SQL), _is_readonly_sql guard	Model drafts SQL; a read-only guard rejects non-SELECT and caps rows (≤1000) before any live execution.
Keep regulated data on-prem	guardrail.py (Layer 0), per-profile data_policy	Single egress chokepoint; default local_only; cloud only when an admin raises the policy with a recorded agreement.
Catalog OnBase DocTypes	zipfile+xml, BU_Catalog.json (Layer 3)	Imports .spk/.expk Studio packages to build a business-value catalog.
Login, roles & permissions	auth.py, SQLite, hmac/hashlib/secrets, permissions.json	HMAC-hashed passwords, token sessions; roles map to tab sets and feature flags.
Reach it remotely	NSSM + Cloudflare Tunnel	QL_MapSnap service behind a named tunnel at the public HTTPS URL.

7. Licensing & compliance posture (summary for IT)

Permissive by default. Almost every third-party component is permissively licensed (MIT / MIT-0 / Apache-2.0 / BSD / public domain), used as an unmodified library or external service. The only copyleft item is psycogp2 (LGPL), which is used as an unmodified, dynamically-linked driver — compliant as shipped.

Data-governance. The egress guardrail enforces FERPA/GLBA/HIPAA posture in code, not by convention: default local_only, optional cloud_metadata_only (structure only, values scrubbed), and locked profiles that require an admin + recorded agreement to raise. Every cloud decision is logged for audit.

On-prem AI. The default AI path is fully local (Ollama + local embeddings); no document content or regulated value reaches a cloud provider unless a profile is explicitly, auditable raised.